everything you need to build web apps, but Phoenix focuses only on serving requests. Phoenix integrates with other key libraries—like Ecto—to create a seamless experience for Elixir web developers.

Requests flow through Phoenix in a similar way to Rails apps. Requests enter the application and run through an `Endpoint` module. The endpoint processes the request and eventually submits it to the `Router` module. Your application router will determine how the request should be processed and then will submit it to a `Controller` module. Finally, your controller executes the request and returns a result to the client. This entire process is explicit in Phoenix, so you can trace exactly how a request is handled by your application.

Controllers accept input from users and determine how to process a request. They coordinate with the rest of your application. It's most common to have controllers call into your application code to do things like read and write from the database. Once a controller has processed a request, it renders a response (HTML, JSON, or other formats) that's returned to the client. Often, view-modules (like `MessageHTML`) are used to render the correct HTML.

The frontend of large applications can quickly become unwieldy. Use Phoenix components to create self-contained components. You can optionally define the attributes that a component supports—this helps ensure you always call a component the right way, with attributes of the correct type. Phoenix comes with a `CoreComponents` module that defines many helpful components for quickly building an application.

Phoenix doesn't care too much about your asset pipeline, but it does come with an esbuild-powered pipeline that includes TailwindCSS. You can swap out the asset pipeline for any other system you prefer. But make sure to hash your assets properly and include your assets in the application-build process.

Phoenix provides some awesome libraries and features that we don't cover in this book. The authentication generator gives you an entire user-based authentication system in a few commands. Phoenix channels provide a foundation to create real-time enabled applications. Finally, Phoenix LiveView provides a totally new paradigm for web development with a real-time web framework that lives on the server.

Phoenix is one of the most important libraries, so you'll get familiar with it as you build more Elixir applications. One thing that it doesn't do is make outbound HTTP requests. In the next chapter, we'll explore how the Req library allows us to perform outbound HTTP requests in our app.